

Aarhus University
Cryptographic Computing
Group 2

Identity-Based Encryption from Boneh-Franklin scheme with
Optimized r -Ate Pairing over $BLS12-381$ curve

Fanni Ferenczi Zsanett
202402635@post.au.dk
Miklos Daniel Vadasz
202402632@post.au.dk

November 26, 2025

Contents

1 Overview of Identity Based Encryption schemes	2
1.1 Building blocks for Boneh-Franklin IBE Scheme	2
1.2 FullIdent: Securing Boneh-Franklin scheme against adaptive chosen ciphertext attack	3
2 Bilinear mapping and Diffie-Hellmann assumptions	5
2.0.1 Security of pairings	5
3 Choosing Pairing-friendly Elliptic Curve	6
3.0.1 Choosing Type-3 elliptic curves	6
3.0.2 The Tate Pairing	7
3.0.3 Barreto-Lynn-Scott Curve	7
3.1 Hashing into BLS12-381 Elliptic Curve	8
3.1.1 Encoding Byte Strings to BLS12-381 Curve	8
3.1.2 Simplified Shallue-van de Woestijne-Ulas Method	9
4 Implementation of Boneh-Franklin based IBE and Results	10
A Test results for Boneh-Franklin based IBE scheme	14
B Equivalent transformations for pairings	17
C Hash-to-curve parameters for curve BLS12-381	18
D Libraries for Pairing-based Cryptography	19
D.1 MIRACL	19
D.1.1 SHA-256 Digest	19

1 Overview of Identity Based Encryption schemes

In his original paper [29] Shamir introduced a public key encryption scheme that resembled an ideal mail system, in which the public key can be an arbitrary string which uniquely identifies the user in a way that he/she cannot later deny and that is readily available to the other party.[29]

Shamir’s original motivation for identity-based encryption was to simplify the certificate, signing, and key management processes with regards for example in a mail system.[9].

Let’s say when *Alice* wants to send a mail to *Bob* at **bob@company.com** she simply encrypts her message using the public key string bob@company.com. In the proposed cryptosystem there is no need for *Alice* to obtain *Bob*’s public key certificate. When *Bob* getting noticed that he received the encrypted mail he contacts a trusted third party called the *Private Key Generator* (denoted as *PKG* furthermore) and after authenticating himself to the *PKG* *Bob* obtains his private key from the *PKG*. [9]

The authentication takes places in the same way *Bob* would authenticate himself to a trusted Certificate Authority (denoted as *CA* furthermore). *Bob* can then therefore read his e-mail sent by *Alice*.

In comparison to an existing secure e-mail infrastructure, with the use of the proposed fully functional identity-based encryption scheme[9] *Alice* can send an encrypted mail to *Bob* even if he has not yet setup his public key certificate. Regarding the system’s performance, now it becomes comparable to the performance of ElGamal encryption[18] in $\mathbb{F}_p^*$. The security of the Boneh-Franklin scheme is analogues to the computational Diffie-Hellman assumption[26], namely the new system has chosen ciphertext security in the random oracle model[5].

Following Boneh-Franklin’s first publication, Identity-based schemes underwent significant development, as can be seen summarized in the *Table 1*. Next to the scenario mentioned in paper [29], IBE can be a good choice in cases where *Chosen Ciphertext Security*[15, 11, 13], *digital signatures*, *secret handshake*, *searching on encrypted data*[AB05, 12], forward secure encryption [14, 8]or *adaptive oblivious transfer* [25] are the desired goal to achieve.

IBE-based constructions			
	Based on pair-ings, $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$	Based on lattices (Learning with errors)	Quadratic Resid-uosity
IBE with <i>Random Oracle</i>	Boneh-Franklin scheme[9], Boneh-Boyek scheme[7], Waters-IBE[32], Gentry-IBE[22], DualSys-IBE[W09]	Trapdoors for hard lattices[23]	Space-efficient IBE[10]
IBE without <i>Ran-dom Oracle</i>	Forward-secure PKE[14]	Delegation of lat-tice basis[16]	<i>future work</i>
Hierarchical IBE	Hierarchical ID-based cryptogra-phy[24]	Delegation of lat-tice basis[16]	<i>future work</i>

Table 1: Overview of Identity-Based cryptographic construction families [6]

Also the authors showed that in their scheme *PKG* can be distributed using standard techniques from threshold cryptography[21], consequently given the *Boneh-Franklin* scheme based IBE, the master-key is never available in a single location.

For the detailed description about distributed MSK (*Master Shared-Key*) please refer to *Section ??*.

1.1 Building blocks for Boneh-Franklin IBE Scheme

An identity-based encryption scheme ε is defined formally by four randomized algorithms: Setup, Extract, Encrypt and Decrypt as follows:

Setup: Takes k security parameter as input, and outputs the params system parameters and master-key. The params system parameters describes the finite message space $\mathcal{M}$ and the finite ciphertext space $\mathcal{C}$. In addition, the params system parameters are publicly known, while the master-key will be known only for the Private Key Generator (*PKG* furthermore. Please note, we call *PKG* as *DKG* in case it generates the master-key via *Multiparty-Computation*).[9] For the implementation of Setup algorithm, please refer to *Code snippet ??* within *Section 4*

Extract: Takes params, master-key and an arbitrary $ID \in \{0, 1\}^*$ as input, and outputs d private key. ID is an arbitrary string that corresponds to the private key d . In addition, the Extract algorithm extracts the private key from the given public key [9]. For the implementation of Extract algorithm, please refer to *Code snippet ??* within *Section 4*.

Encrypt: The Encrypt algorithm takes params, ID and $M \in \mathcal{M}$ as input, and outputs $C \in \mathcal{C}$ [9]. For the implementation of Encrypt algorithm, please refer to *Code snippet ??* within *Section 4*.

Decrypt: The Decrypt algorithm takes params and private key d and $C \in \mathcal{C}$ as input, and outputs $M \in \mathcal{M}$ [9]. For the implementation of Decrypt algorithm, please refer to *Code snippet ??* within *Section 4*.

The standards consistency check for the outlined algorithms can be summarized as follows:

$$\forall M \in \mathcal{M} : \text{Decrypt}(\text{params}, \text{Encrypt}(\text{params}, ID, M), d) = M \quad (1)$$

The standard acceptable notion for security is *chosen ciphertext security* (denoted as *IND-CCA* furthermore), however for Identity-Based Encryption schemes the definition can be strengthened a bit for the following reason: when an *Adversary* attacks a *public key ID* then we should also assume that the *Adversary* might already have obtained the private key associated with any identity ID_i , other than the public key ID being attacked. We refer such queries as *private/public key extraction queries* furthermore. [9]

We say that the identity-based decryption scheme ε is semantically secure against an adaptive chosen ciphertext attack (*IND-ID-CCA*) if no polynomially bounded adversary $\mathcal{A}$ has a non-negligible advantage against the *Challenger* in the following *IND-ID-CCA* game: During the *Setup* phase the challenger takes a security parameter k and runs the Setup algorithm, then it keeps the master-key to itself and sends the resulting system parameters params to $\mathcal{A}$ *Adversary* [9]. In *Phase 1* the adversary issues private key extraction queries $ID_1, \dots, ID_m$. The challenger responds by running algorithm Extract to generate the private key d_i corresponding to the public key ID_i . It sends d_i to the adversary. The queries may be asked adaptively. Throughout the *Challenge phase* the adversary outputs two equal length plaintexts $M_0, M_1 \in \mathcal{M}$ and a public key ID (such that it did not appear in any private key extraction query in *Phase 1*) it wishes to be challenged. The challenger picks a random bit $b \in \{0, 1\}$ and sets $C = \text{Encrypt}(\text{params}, ID, M_b)$. Then C will be sent as a challenge to the adversary. In *Phase 2* the adversary issues more extraction queries $ID_{m+1}, \dots, ID_n$ with constraint $ID_i \neq ID$. The challenger responds as in *Phase 1*. Finally, the adversary outputs the guess $b' \in \{0, 1\}$ and wins the game if $b = b'$. [9]

Please note, for the rest of our report we use $\mathbb{Z}_q$ to denote the group $\{0, \dots, q-1\}$ under addition modulo q . Therefore a group $\mathbb{G}$ of prime order we use $\mathbb{G}^*$ which denotes the set $\mathbb{G}^* = \mathbb{G} \setminus \{O\}$, where O denotes the identity element of group $\mathbb{G}$. To denote the set of positive integers we will use $\mathbb{Z}^+$ in accordance with the notation used in [9].

1.2 FullIdent: Securing Boneh-Franklin scheme against adaptive chosen ciphertext attack

The authors also extended their basic identity-based encryption to be secure against an adaptive chosen ciphertext attack (*IND-ID-CCA*) in the random oracle model. [9] The scheme is incorporating four algorithms: *Setup*, *Extract*, *Encrypt* and *Decrypt* as can be followed within *FullIdent algorithm 1*.

In every case the message space is $\mathcal{M} = \{0, 1\}^n$ and cipherspace is $\mathcal{C} = \mathbb{G}_1^* \times \{0, 1\}^n$. Also $\mathcal{G}$ denotes some BDH parameter generator (for further the description of bilinear maps and the underlying Diffie-Hellman Assumptions refer to *Section 2*). [9]

Setup phase: ;

Input: k , security parameter chosen from $\mathbb{Z}^+$

Output: System parameters $\text{params} = \langle q, \mathbb{G}_1, \mathbb{G}_2, \hat{e}, n, P, P_{\text{pub}}, H_1, H_2 \rangle$

Step 1: We run $\mathcal{G}$ on k which yields a prime q , $\mathbb{G}_1, \mathbb{G}_2$ groups of order q , and an admissible bilinear map $\hat{e} : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}$. Random generator is chosen $P \in \mathbb{G}_1$.

Step 2: We pick a random $s \in \mathbb{Z}_q^*$ then set $P_{\text{pub}} = sP$, where s is the master-key $s \in \mathbb{Z}_q^*$

Step 3: Cryptographic hash H_1, H_2, H_3, H_4 functions are chosen as random oracles:

$H_1 : \{0, 1\}^* \rightarrow \mathbb{G}_1^*$

$H_2 : \mathbb{G}_2 \rightarrow \{0, 1\}^n$ for some n .

$H_3 : \{0, 1\}^n \times \{0, 1\}^n \rightarrow \mathbb{Z}_q^*$

$H_4 : \{0, 1\}^n \rightarrow \{0, 1\}^n$

Extract phase: ;

For given $ID \in \{0, 1\}^*$ string we compute $Q_{id} = H_1(ID) \in \mathbb{G}_1^*$ and setting d_{id} private key to be $d_{ID} = sQ_{ID}$, where s is the master key as before.

Encrypt phase: ;

To encrypt $M \in \{0, 1\}^n$ with public key ID , we compute: $Q_{ID} = H_1(ID) \in \mathbb{G}_1^*$, then choose a random $\sigma \in \{0, 1\}^n$, then set $r = H_3(\sigma, M)$ lastly the C ciphertext can be computed as:

$C = \langle rP, \sigma \oplus H_2(g_{ID}^r), M \oplus H_4(\sigma) \rangle$, where $g_{ID} = \hat{e}(Q_{ID}, P_{\text{pub}}) \in \mathbb{G}_2$.

Decrypt phase: ;

Let $C = \langle U, V, W \rangle$ be the ciphertext encrypted with public key ID . First we check $U \neq \mathbb{G}_1^*$ holds, if true we reject the C ciphertext. To decrypt C with private key $d_{ID} \in \mathbb{G}_1^*$ we proceed as follows:

1. Compute $V \oplus H_2(\hat{e}(d_{ID}, U)) = \sigma$
2. Compute $W \oplus H_4(\sigma) = M$.
3. Set $r = H_3(\sigma, M)$. Test that $U = rP$, if not reject the ciphertext.
4. Output M as the decryption of C .

Algorithm 1: Boneh-Franklin based scheme: FullIdent Algorithm

For consistency verification, we make sure that during encryption M is bitwise XOR -ed with the hash of g_{ID}^r . Also, during encryption V is bitwise XOR -ed with the hash of $\hat{e}(d_{ID}, U)$. We use the properties of pairings for checking these masks are the same as follows [9]:

$$\hat{e}(d_{ID}, U) = \hat{e}(sQ_{ID}, rP) = \hat{e}(Q_{ID}, P)^{sr} = \hat{e}(Q_{ID}, P_{\text{pub}})^r = g_{ID}^r \quad (2)$$

To convert the *BasicIdent*[9] scheme into a chosen ciphertext secure system *Fujisaki-Okamoto*'s technique can be used in the random oracle model as proposed within [20]. Let $\varepsilon_{pk}(M; r)$ be the encryption of M using r random bits with pk public key. Therefore the *Fujisaki-Okamoto* hybrid scheme can be formalized in Equation 3

$$\varepsilon_{pk}^{hy}(M) = \langle \varepsilon_{pk}(\sigma; H_3(\sigma, M)), H_4(\sigma) \oplus M \rangle \quad (3)$$

Within Equation 3 H_3, H_4 denotes cryptographic hash functions, and σ is generated at random. In [20] Fujisaki-Okamoto showed that if ε is a one-way encryption scheme then ε^{hy} is a chosen ciphertext secure system (IND-CCA) in the random oracle model (assuming ε_{pk} satisfies some natural constraints). In addition, semantic security implies one-way encryption and hence the Fujisaki-Okamoto result also applies if ε is semantically secure (IND-CPA). For instance, M is encrypted as $W = M \oplus H_4(\sigma)$ within *FullIdent Algorithm 1*, but it can be replaced by $W = E_{H_4(\sigma)}(M)$, with respect to semantically secure symmetric encryption scheme of E . [9]

To see that *FullIdent* is chosen ciphertext secure *IBE* (IND-ID-CCA) under BDH assumption (BDH is hard within groups generated by $\mathcal{G}$) we use *Theorem 4.4* from [9].

Given an IND-ID-CCA adversary $\mathcal{A}$ that has advantage $\varepsilon(k)$ against the *FullIdent* scheme with running time at most $t(k)$. Then $\mathcal{A}$ makes at most q_E extraction and at most q_D decryption queries, while at most $q_{H_2}, q_{H_3}, q_{H_4}$ queries for the hash functions H_2, H_3, H_4 respectively. Thus the BDH algorithm for $\mathcal{G}$ with running time at most $t(k)$ can be written[9] as follows:

$$\text{Adv}_{\mathcal{G}, \mathcal{B}}(k) \geq 2FO_{adv} \left(\frac{\varepsilon(k)}{\varepsilon(1 + q_E + q_D)} \right) q_{H_2} \quad (4)$$

$$t_1(k) \leq FO_{time}(t(k), q_{H_4}, q_{H_3}) \quad (5)$$

2 Bilinear mapping and Diffie-Hellmann assumptions

Let $\mathbb{G}_1$ and $\mathbb{G}_2$ be two groups of order q for the prime q . In order to have $\hat{e} : \mathbb{G}_1 \rightarrow \mathbb{G}_2$ as an admissible bilinear map for *BF-IBE* system, *bilinearity*, *non-degeneracy*, *computability* properties must be satisfied as follows. The map $\hat{e} : \mathbb{G}_1 \times \mathbb{G}_2$ satisfies bilinearity if $\hat{e}(aP, bQ) = \hat{e}(P, Q)^{ab}$ for all $P, Q \in \mathbb{G}_1$ and for all $a, b \in \mathbb{Z}$. Non-degenerate property means that $\hat{e}$ map does not send all pairs in $\mathbb{G}_1 \times \mathbb{G}_1$ to the identity $\mathbb{G}_2$, this also implies if P is a generator of $\mathbb{G}_1$ then $\hat{e}(P, P)$ is a generator of $\mathbb{G}_2$. *Computability* means that some efficient algorithms exists to compute $\hat{e}(P, Q)$ for any $P, Q \in \mathbb{G}_1$. [9]

For a more broader list of equivalent transformations for pairing please refer to *Appendix Section B*.

2.0.1 Security of pairings

Later in our report we choose a specific E elliptic curve family in conjunction to pairings (please refer to *Section ??*), hereby we note the summarization of security considerations related to $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ pairings based on [3]: The *(EC)DLP* problem must be hard in $\mathbb{G}_1, \mathbb{G}_2$ and $\mathbb{G}_T$, hardness of $\mathbb{G}_T$ is given by $k \cdot |q|$. When choosing parameters in $\mathbb{G}_1, \mathbb{G}_2$ we must ensure they are large enough. To approximate an optimal choice ρ describes the balance between r, q parameters: $\rho = \frac{\log q}{\log r}$. ($\rho = 1$ is optimal).

We choose k security parameter in accordance of RFC9380[19] standard, while for implementation of building and computing the map $\hat{e}$ we chose MIRACL[30] specific libraries.

The existence of $\hat{e} : \mathbb{G}_1 \times \mathbb{G}_1 \rightarrow \mathbb{G}_2$ mapping implies that the discrete log problem in $\mathbb{G}_1$ can be reduced to a discrete log problem in $\mathbb{G}_2$, with *MOV* (Menezes, Okamoto, and Vanstone) reduction method.[9] Therefore we must choose the security parameter in a way that discrete log is hard in $\mathbb{G}_2$.

On the otherhand the Decision Diffie-Hellman (noted as *DDH* furthermore) is easy in $\mathbb{G}_1$ since we can distinguish between the distributions $\langle P, aP, bP, abP \rangle$ and $\langle P, aP, bP, cP \rangle$ with $a, b, c \in \mathbb{Z}_q^*$ and $P \in \mathbb{G}_1^*$ as *Joux* and *Nguyen* pointed out. [9]

For this reason the Boneh-Franklin IBE scheme builds upon the Computational Diffie-Hellmann assumption, also called *Bilinear Diffie-Hellman Assumption* (noted as *BDH* furthermore): Given our $\mathbb{G}_1, \mathbb{G}_2$ and $\hat{e}$ mapping and $\langle P, aP, bP, cP \rangle$ for some $a, b, c \in \mathbb{Z}_q^*$ compute $W = \hat{e}(P, P)^{abc} \in \mathbb{G}_2$. Then an algorithm $\mathcal{A}$ has advantage ϵ in solving *BDH* for $\langle \mathbb{G}_1, \mathbb{G}_2 \rangle$ if:

$$\Pr [\mathcal{A}(P, aP, bP, cP) = \hat{e}(P, P)^{abc}] \geq \epsilon \quad (6)$$

with probability where $a, b, c \in \mathbb{Z}_q^*$ and $P \in \mathbb{G}_1^*$ are random with random bits of $\mathcal{A}$. [9]

Previously we used $\mathcal{G}$ as *BDH* generator within *Section 1.2*, we give the full description of the assumption here based on [9]: Having $\mathbb{G}$ be a *BDH* parameter generator, than an $\mathcal{A}$ algorithm has advantage $\epsilon(k)$ in solving the *BDH* problem for $\mathbb{G}$ for sufficient large k [9]:

$$\begin{aligned} Adv_{\mathcal{G}, \mathcal{A}}(k) = \Pr \left[\mathcal{A}(q, \mathbb{G}_1, \mathbb{G}_2, \hat{e}, P, aP, bP, cP) = \hat{e}(P, P)^{abc} \mid \langle g, \mathbb{G}_1, \mathbb{G}_2, \hat{e} \rangle \leftarrow \mathbb{G}(1^k), \right. \\ \left. P \leftarrow \mathbb{G}_1^*, a, b, c \leftarrow \mathbb{Z}_q^* \right] \geq \epsilon(k) \end{aligned} \quad (7)$$

Consequently $\mathbb{G}$ satisfies the *BDH* assumption for any randomized algorithm $\mathcal{A}$ has negligible advantage in k for $Adv_{\mathbb{G}, \mathcal{A}}(k)$. In the following section we briefly review necessary background for pairing-based cryptography and how the aforementioned bilinear mapping can help us to choose the right elliptic curve family for our *BF-IBE* implementation.

3 Choosing Pairing-friendly Elliptic Curve

An elliptic curve E is specified by an equation in two variables and a finite field $\mathbb{F}$. The equation that describes the elliptic curve can take standard forms, including (but not limited to) *Weierstrass*, *Montgomery*, and *Edwards*. In addition the curve E induces an algebraic group of order n , meaning that the group has n distinct elements. [19] Please note, in this report we use additive notation for the elliptic curve group operation.

An elliptic curve is said to be the set of solutions to the *Weierstrass form equation 8* with respect to $(x, y) \in \mathbb{F}_{q^m} \times \mathbb{F}_{q^m}$. [3] The elements of an elliptic curve group are points with affine coordinates (x, y) satisfying the curve equation, where x and y are elements of $\mathbb{F}$. In addition, all elliptic curve groups have a distinguished element, called the *identity point*, which acts as the identity element for the group operation. (In case of *Weierstrass* and *Montgomery* curves, the identity point cannot be represented as an (x, y) coordinate pair.) [19]

With regards to security reasons we should take into consideration that cryptographic applications of elliptic curves generally require using a (sub)group of prime order, therefore let G denotes the subgroup of the curve of prime order r with respect to h cofactor, where $n = h \cdot r$. We also note, that an algorithm that as input an arbitrary point on the curve E and produces as output a point in the subgroup G of E is often referred as *cofactor clearing*. Also, certain hash-to-curve algorithms restrict the form of the curve equation, the characteristic of the field, or the parameters of the curve. [19] Please refer to *Section 3.1* for further details.

In accordance with *RFC9380* for this report we use notation the relevant to elliptic curves and quantities relevant to hashing to curves (refer to later *Section 3.1*) introduced within [19] as follows: E denotes elliptic curve (for instance E is specified by *8 equation* and a field $\mathbb{F}$). $\mathbb{F}, p, q$ denotes finite field of $\mathbb{F}$ of characteristics p and $\#F = q = p^m$ in case of prime fields $q = p$, otherwise $q = p^m$ and $m > 1$. n will be the number of points on the E elliptic curve computed as $n = h \cdot r$. In the equation r denotes the order of G , also it is (usually) the largest prime factor of n . h is the so called cofactor, with property $h \geq 1$ and $n = h \cdot r$. G denotes a prime-order subgroup of the points on E , consequently G is a destination group to which byte strings are encoded. [19]

$$E : y^2 = x^3 + ax + b, \quad (8)$$

where $a, b \in \mathbb{F}_{q^m}$ $\Delta \neq 0$ (discriminant) and a point at infinity (∞). [3] Also E' a degree d twist of E is an isomorphic curve to E over the algebraic closure of $\mathbb{F}_{q^m}$. The only possible degrees for elliptic curves are $d \in 2, 3, 4, 6$. [3] The order n of the curve is the number of points that satisfy the curve equation. The *Hasse coddition* states the following *9 equation*:

$$n = q^m + 1 - t, \quad |t| \leq 2\sqrt{q^m} \quad (9)$$

The literature calls such curves *supersingular* if q divides t . The order of point P is the smallest integer r such that $rP = \infty$. It is also true that the order of the point is a dividant of the curve order: $r|n$. The r -torsion subgroup (denoted as $E(\mathbb{F}_{q^m}[r])$) is the set of points P in which their order divides r . [3]

3.0.1 Choosing Type-3 elliptic curves

In case of pairings ($e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$) there are some properties we can rely on. Namely, $\mathbb{G}_1$ is typically a subgroup of $E(\mathbb{F}_q)$, $\mathbb{G}_2$ is typically a subgroup of $E(\mathbb{F}_q^k)$, lastly $\mathbb{G}_T$ is a multiplicative subgroup of $\mathbb{F}_{q^k}^*$. [3]

The embedding degree of $\mathbb{F}_q$ is the smallest integer k , in which we can embed the r -torsion group: $r|(q^k - 1)$. For efficiency reasons we want to the largest d such that $d|k$. [3] The families of ordinary curves usually have $k < 50$, random curves have $k \approx q$, but for supersingular curves we have embedding degree of $k \leq 6$. [3]

From the *Adversary's* point of view k is usually very large and hence mapping the problem into the $\mathbb{F}_{p^k}$ field is useless for an attacker. As for super-singular curves that the k embedding degree is small, but on the other hand, If we choose the curves large enough, discrete log will still be hard despite the attacks. [17]

Additionally, when generating curves we also have to be cautious about the $\mathbb{G}_i$ relations. In case $\mathbb{G}_1 = \mathbb{G}_2$ the pairing is symmetric (also called *Type-1* curves) and defined over a supersingular curve with ψ distorsion map as can be seen in *Equation 10* [3]:

$$\psi : E(\mathbb{F}_q[r]) \rightarrow E(\mathbb{F}_q^k[r]) \quad (10)$$

With assymmetric (also called *Type-3* curves) $\mathbb{G}_1 \neq \mathbb{G}_2$, and $\mathbb{G}_2$ is chosen as the group of points in the twist that is isomorphic to a subgroup of $E(\mathbb{F}_q^k[r])$. In the latter case there is no efficient map $\psi : \mathbb{G}_2 \rightarrow \mathbb{G}_1$. [3]

Let p be a prime and E be the elliptic curve defined over $\mathbb{F}_p$ finite field. Let r be a prime with $r \mid \#E(\mathbb{F}_p)$ and $\gcd(r, p) = 1$. Then the cofactor is $\rho = \frac{\log p}{\log q}$. Let k be the *embeddding degree*, which k is the smallest positive integer with $r|(p^k - 1)$. Let $\Pi : (x, y) \rightarrow (x^p, y^p)$ be the p -th power *Frobenius* endomorphism. The trace of the *Frobenious* is $t = p + 1 - \#E(\mathbb{F}_p)$. [2]

Then $\mathbb{G}_1$ can be rewritten as the 1-eigenspace of Π acting on $E[r]$ as follows: $\mathbb{G}_1 = \{P \in E[r] : \Pi(P) = P\} = E(\mathbb{F}_p)[r]$. Also let d be the order of the automorphism group of E in a way that $d|k$ also holds.[2]

Let $e = k/d$ and $q = p^e$. Then the unique degree- d twist (denoted as $\tilde{E}$) of E over $\mathbb{F}_q$ has the property of $r|\#\tilde{E}(\mathbb{F}_q)$. The associated twisting isomorphism can be described as $\Psi : \tilde{E} \rightarrow E$. Let $\tilde{Q} \in \tilde{E}(\mathbb{F}_q)$ be the point of order r then $Q = \Psi(\tilde{Q}) \notin E(\mathbb{F}_p)$. Then we can view $\mathbb{G}_2$ as the p -eigenspace of Π acting on $E[r]$, written as $\mathbb{G}_2 = \langle Q \rangle$. Finally, $\mathbb{G}_T$ denotes the order- r subgroup of $\mathbb{F}_{p^k}^*$. [2]

The literature calls this non-degenerate bilinear maps from $\mathbb{G}_1 \times \mathbb{G}_2$ to $\mathbb{G}_T$ as Type 3 pairings. [2]

3.0.2 The Tate Pairing

Let E be an elliptic curve containing n points over the field $\mathbb{F}_q$, G will denote a cyclic subgroup of $E(\mathbb{F}_q)$ of order r with r, q coprime. Let k be the smallest positive integer such that $r|q^k - 1$, while K be the smallest extension of $\mathbb{F}_q$ containing the r th roots of unity as $K = \mathbb{F}_{q^k}$. Then the Tate pairing $e : E[r] \cap E(K) \times E(K)/rE(K) \rightarrow K^*/K^{*r}$. [27]

Let f_P be a rational function with divisor $(f_P) = (P)^r$, then choose $R \in E(K)$ such that $R \neq P, P - Q, O, -Q$. Then we can define $f(P, Q) = f_P(Q + R)/f_P(R)$. It can be showed that the value is independent of the choice of R , therefore $f(aP, bQ) = e(P, Q)^{ab}$ for all P, Q, a, b in case of $f(P, Q) = 1$ for all P if $Q = O$. Consequently $f(P, Q) = 1$ for all Q if $P = O$. With regards to the Frobenius map we have $f(\Phi(P), \Phi(Q)) = f(P, Q)^q$ for all $P, Q \in E[r]$ where Φ denotes the Frobenius map. Lastly the output of the pairing scheme is some $x \in K^*$ that represents the coset xK^{*r} . [27]

We can now view our scenario as a special case of $R = O$, thus the intermediate function f_k can be defined by $(f_k) = (P^k)/(kP)$. This yields us with equality $(f_r) = (f_P)$. Now $f_{k+1} = (f_k L_{kP, P}/V_{(k+1)P})$ gives us Miller's algorithm as can be seen *Algorithm 2*. Miller's algorithm computes $f_P(Q)$ given points P, Q with respect to r order of P . Finally it outputs $x = f_r(Q)$, with respect to $Z = rP = O$ [27]

```

x ← 1
Z ← P
for i ← t - 1, ... 0 do
    x ← x2 · TZ(Q)/V2Z(Q)
    Z ← 2Z if ri = 1 then
        x ← x · LZ, P(Q)/VZ+P(Q)
        Z ← Z + P
    end
end
end

```

Algorithm 2: Miller's algorithm for Tate pairing $x = f_P(Q)$

For our project we are rely upon MIRACL[30] `ate()` implementation which uses Optimal r -Ate pairing (refer to *Section ??*). Although we note that there are many different optimization techniques have been researched and implemented with regards to pairings [4] as well.

3.0.3 Barreto-Lynn-Scott Curve

As we reviewed previously pairing is a kind of bilinear map defined over two elliptic curves E and E' . A Barreto-Naehrig Curve (BN) curve [BN05] is one of the instantiations of pairing-friendly curves proposed in 2005. A pairing over BN curves constructs optimal Ate pairings. In contrast to BN curves, $E(F_p)$ does not have a prime order, instead its order is divisible by a large parameterized prime r and denoted by $h * r$ with cofactor h . The pairing will be defined on the r -torsions points.[28]

More formally a BLS curve family consideres elliptic curves E and E' parameterized by a well chosen integer t . E is defined over a finite field F_p by an equation of the form $E : y^2 = x^3 + b$, and its twisted curve, $E' : y^2 = x^3 + b'$, where b is an element of multiplicative group of order p . Similarly to BN curves BLS curves can also be categorized as D-type and M-type whether $b' = b/m$ or $b' = b * m$ respectively.[28]

The BLS curve can be parameterized with p, r with respect to t as follows[28]:

$$p = (t - 1)^2 \cdot (t^4 - t^2 + 1)/3 + t \quad (11)$$

$$r = t^4 - t^2 + 1 \quad (12)$$

Then the $\hat{e}$ pairing can be defined by taking $\mathbb{G}_1$ as a subgroup of $E(\mathbb{F}_p)$ of order r , $\mathbb{G}_2$ as an order r subgroup of $E'(\mathbb{F}_{p^2})$, lastly $\mathbb{G}_T$ as an order r subgroup of a multiplicative group $(\mathbb{F}_{p^2})^*$. [28]

For our implementation we choosed embedding degree as 12 thus BLS12 curve family was choosen. Secondly we aimed at 128-bit security level to be close as possible for RFC9380 compliant regarding the hashing to curve

problem. Although we aware of that attacks such as extended tower number field sieve (exTNFS), have been proposing to aim for higher security levels with regards to choosing the minimum bit length of p as 384 bits[28], we choosed to use the standardized parameters and test vectors for BLS-381.

Thus, t is defined by $t = -2^63 - 2^62 - 2^60 - 2^57 - 2^48 - 2^16$, and size of p becomes 381-bit length. The E elliptic curve its twisted curve E' are represented by $E : y^2 = x^3 + 4$ and $E' : y^2 = x^3 + 4(u + 1)$, the $\hat{e}$ is defined by taking $\mathbb{G}_1$ as a cyclic group of order r generated by a base point $BP = (x, y)$ in $\mathbb{F}_p$, $\mathbb{G}_2$ as a cyclic group of order r generated by a based point $BP' = (x', y')$ in $\mathbb{F}_{p^2}$, and $\mathbb{G}_T$ as a subgroup of a multiplicative group $(\mathbb{F}_{p^{12}})^*$ of order r .

For the finite field $\mathbb{F}_p$, the towers of extension field $\mathbb{F}_{p^2}$, $\mathbb{F}_{p^6}$ and $\mathbb{F}_{p^{12}}$ are defined by indeterminates u, v, w as follows:

$$\mathbb{F}_p^2 = \mathbb{F}_p[u]/(u^2 + 1) \quad (13)$$

$$\mathbb{F}_p^6 = \mathbb{F}_p^2[v]/(v^3 - u - 1) \quad (14)$$

$$\mathbb{F}_p^{12} = \mathbb{F}_p^6[w]/(w^2 - v) \quad (15)$$

For the parameters specified by [28] please refer to *Appendix Section ??*.

3.1 Hashing into BLS12-381 Elliptic Curve

To analyze the security of cryptographic constructions Bellare and Rogaway introduced an idealized security model called the *Random Oracle Model* [5]. We can think about such random oracle as a function defined das $H : X \rightarrow Y$. Chosen uniformly at random from the set of all functions $\{h : X \rightarrow Y\}$ with assumption of Y is a finite set. The algorithm that queries the random oracle then receive the value $H(x)$ for all $x \in X$ in response. [9]

Many cryptographic protocols require a procedure that encodes an arbitrary input, e.g., a password, to a point on an elliptic curve. This procedure is known as hashing to an elliptic curve, where the hashing procedure provides collision resistance and does not reveal the discrete logarithm of the output point. [19]

Since the precise hash function that is suitable for a given protocol implemented using a given elliptic curve is often unclear from the protocol's description and probabilistic rejection sampling methods (referred as *try-and-increment*) are not recommended because of side-channel vulnerabilities *BLS12-381* suite was choosed in accordance to RFC 9380, with regards to serialization, domain separation, `expand_message_xmd` function and finally with hashing suite *BLS12381G1_XMD:SHA-256_SSWU*. [19]

3.1.1 Encoding Byte Strings to BLS12-381 Curve

The general constructions for encoding byte strings to points on an elliptic curve relies on three functions: `hash_to_field` hashes arbitrary-length byte strings to a list of one or more elements of a finite field $\mathbb{F}$, function `map_to_curve` calculates a point on the elliptic curve E from an element of the finite field $\mathbb{F}$ over which E is defined, lastly the function `clear_cofactor` sends any point on the curve E to the subgroup G of E . [19]

The two distinct Algorithm for encodings (`encode_to_curve()`, `hash_to_curve()`) are both have the same interface and are both random-oracle encodings. The only difference between them is their outputs are sampled from different distributions. The algorithm `encode_to_curve()` is a a nonuniform encoding from byte strings to points in G , meaning that the distribution of its output is not uniformly random in G . Consequently the set of possible outputs of `encode_to_curve()` is only a fraction of the points in G , and some points in this set are more likely to be output than others as can be followed in *pseudo-code 3*. [19]

Input: *msg*, an arbitrary-length byte string

Output: P , a point in G

Steps: ;

1. $u = \text{hash_to_field}(msg, 1)$;
2. $Q = \text{map_to_curve}(u[0])$;
3. $P = \text{clear_cofactor}(Q)$;
4. return P ;

Algorithm 3: Pseudocode for encoding into elliptic curve[19] under Random-Oracle Model

On the other hand, `hash\_to\_curve()` is a uniform encoding from byte strings to points in G , as can be followed in *pseudo-code 4*, where $R = Q0 + Q1$ represents point addition on the curve. Meaning that the distribution of its output is statistically close to uniform in G , consequently the function is suitable for applications requiring a random oracle returning points in G , when instantiated with any of the mapping onto curve functions, in accordance to RFC 9380[19].

Input: msg , an arbitrary-length byte string

Output: P , a point in G

Steps: ;

1. $u = \text{hash_to_field}(msg, 2)$;
2. $Q0 = \text{map_to_curve}(u[0])$;
3. $Q1 = \text{map_to_curve}(u[1])$;
4. $R = Q0 + Q1$;
5. $P = \text{clear_cofactor}(R)$;
6. return P ;

Algorithm 4: Pseudocode for hashing to elliptic curve[19] under Random-Oracle Model

The Table 2 within Appendix Section ?? summarizes the parameters for the chosen BLS12-381 curve with regards to G_1, G_2 specified by RFC9380[19].

When hashing passwords using any function an adversary who learns the output of the hash function (or potentially any intermediate value, e.g., the output of `hash_to_field`) may be able to carry out a dictionary attack.[19]

To mitigate such attacks, we note that execution a more costly key derivation function (e.g., *PBKDF2*, *scrypt*, or *Argon2*) on the password, then hashing the output of that function to the target *BLS12-381* elliptic curve in accordance with Section 5.3.1. of [19] can be used as a further extension of our solution. [19]

3.1.2 Simplified Shallue-van de Woestijne-Ulas Method

In case of hash-to-curve methods *probabilistic rejection sampling* (also referred as "try-and-increment" or "hunt-and-peck") is not recommended, because they have been a perennial cause of side-channel vulnerabilities.[19].

Hence we use function `map_to_curve_simple_swu(u)` and the specific libraries from *MIRACL* that implements a simplification of the Shallue-van de Woestijne-Ulas mapping. Please refer to Appendix Section for specification and class descriptions from such libraries.

Wahby and Boneh [WB19] showed how to adapt the Simplified SWU mapping to Weierstrass curves having $A == 0$ or $B == 0$. As a prerequisite this method requires to found such E' by $y'^2 = g'(x') = x'^3 + A' * x' + B'$ that is isogenous to E and has $A' \neq 0$ and $B' \neq 0$. This isogeny defines a map $iso_map(x', y')$ given by a pair of rational functions. iso_map takes as input a point on E' and produces as output a point on E . Once E' and iso_map are identified, this mapping works as follows: on input u , first apply the Simplified SWU mapping to get a point on E' , then apply the isogeny map to that point to get a point on E . In addition iso_map is a group homomorphism, meaning that point addition commutes with iso_map . Thus, when using this mapping in the *hash_to_curve* construction one can effect a small optimization by first mapping $u0$ and $u1$ to E' , adding the resulting points on E' , and then applying iso_map to the sum. This gives the same result while requiring only one evaluation of iso_map . [19]

4 Implementation of Boneh-Franklin based IBE and Results

For elliptic curve point multiplication, there are potentially exploitable side-channel attacks that arise from using the classic *double-and-add* algorithm and its variants. Vulnerabilities arise if branches are taken that depend on secret bits, or if data is even accessed using secret values as indices.[[lib.miracl.report](#)]

As a counter-measure the simplest solution is to use a constant-time algorithm like the *Montgomery ladder*, which uses very little memory and has no key-bit-dependent branches. Other suggestions for representation may be the *fixed-sized signed window method*. To tackle down the pairings related hash-to-curve problems and the previously mentioned reasons we choosed to use MIRACL Core library[[30](#)].

MIRACL Core has built-in support for most standardised elliptic curves, along with many curves that have been proposed for standardisation. As we previously mentioned is [Section 3.0.1](#) BN and BLS curves are specific elliptic curves whose supporting pairing based cryptography. These are the so-called Type-3 pairings. The MIRACL Core library provides optimized functions for point multiplications in the elliptic curve groups $\mathbb{G}_1, \mathbb{G}_2$ and exponentiation in a finite extensions field $\mathbb{G}_T$ and for the pairing itself.[[lib.miracl.report](#)]

The *MIRACL-core*[[30](#)] implementation of Identity-Based Encryption (IBE) utilizes the BLS12-381 pairing-friendly elliptic curve to allow encryption using arbitrary identity strings such as email addresses. The core cryptographic architecture is organized into four classes: **Setup**, **Extract**, **Encrypt**, and **Decrypt**, along with the supporting **Ciphertext** class. The **Setup** class initializes the cryptographic environment by generating the elliptic curve groups $\mathbb{G}_1, \mathbb{G}_2$, and the target group $\mathbb{G}_T$, selecting generators $P \in \mathbb{G}_1$ and $Q \in \mathbb{G}_2$, and generating a master secret s . From this secret, the corresponding public keys $P_{pub} = sP$ and $Q_{pub} = sQ$ are derived. Additionally, **Setup** provides utility functions such as hashing identity strings to curve points, converting pairing results into byte arrays, deriving random exponents, and generating message masks. For instance, the hash-to-curve operation is implemented via `hashToG1`[[30](#)]:

```
ECP Q_ID = hashToG1(identity);
```

Listing 1: Mapping identity string (hash) to $\mathbb{G}_1$ using ECP.java class

which deterministically maps an identity string to a point on the elliptic curve. The **Extract** class uses this mapping to compute the private key for a given identity by multiplying the hashed identity point by the master secret:

```
ECP d_ID = PAIR.G1mul(Q_ID, masterKey.s);
```

Listing 2: Pairing computation using *G1mul()* method from ECP java class

The resulting **PrivateKey** object contains both the identity and its corresponding private key point.

The **Encrypt** class performs the encryption of messages intended for a specific recipient. For each message, a random byte array σ is generated, which is used along with the message to compute a random exponent $r = H_3(\sigma, M)$. This exponent is used to compute $U = r \cdot P$, which forms part of the ciphertext. The pairing mask is then calculated as $g_{ID} = e(Q_{pub}, Q_{ID})^{H(U)}$, and the ciphertext components $V = \sigma \oplus H_2(g_{ID}^r)$ and $W = M \oplus H_4(\sigma)$ are derived. The complete ciphertext is encapsulated in a **Ciphertext** object:

```
Ciphertext C = new Ciphertext();
C.U = U;
C.V = V;
C.W = W;
C.recipientIdentity = identity;
```

Listing 3: Masking operations within the Ecrypt.java class

Decryption is handled by the **Decrypt** class. Given a **Ciphertext** and a **PrivateKey**, the pairing between U and the private key is computed, from which σ is recovered. The original message is then reconstructed by XORing the recovered σ with W , and the correctness is verified using the equality $U = H_3(\sigma, M) \cdot P$. This ensures chosen-ciphertext security and detects any tampering with the ciphertext:

```
// Verification during decryption
BIG r = hashToZq(sigma, M, params.q);
ECP expected_U = PAIR.G1mul(params.P, r);
if (!ciphertext.U.equals(expected_U)) return null;
```

Listing 4: Verification of Chosen-ciphertext security during encryption

A procedure related to encoding is the conversion of an elliptic curve point to a bit string is called serialization. Typically it is used for compactly storing or transmitting points, as its inverse operation, deserialization, converts a bit string to an elliptic curve point. During deserialization the only difference from encoding is that only

certain strings (namely, those output by the serialization procedure) can be deserialized. [19] The `Ciphertext` class supports serialization and deserialization to facilitate secure transmission between parties:

```
byte[] serialized = C.toBytes();
Ciphertext restored = Ciphertext.fromBytes(serialized, identity);
```

Listing 5: serialization and deserialization

In addition the implementation enforces cross-identity security, as each identity receives a unique private key, ensuring that only the intended recipient can decrypt a message. Randomization is achieved by generating a fresh σ for every encryption, so encrypting the same message multiple times produces distinct ciphertexts. Comprehensive testing verifies correctness of encryption and decryption, proper rejection of invalid keys, tamper detection for altered ciphertext components, and the uniqueness of ciphertexts for repeated encryptions. Lastly the *Picture 1* showcases the output of the successful functionality test. For the full result log please refer to *Appendix Section ??*

```
=====
TESTING COMPLETE ENCRYPT/DECRYPT CYCLE (MIRACL-CORE)
=====

=== Extract Algorithm ===
Generating private key for identity: bob@companyEU.com
Step 1: Q_ID = H1(ID) computed
Q_ID: (009e9db06e3d3c7934d088bf37b8d78c05cce8ca1f24b91d4b38145f02b81e968fd14df98e76d152d4a88429a3852a42,11be4647ba3056cc6821ee30cef4b2fd024d6942934a9b002f6e1fe6627d5fda14b3ba8e1c37d6a4ec9766c0da985ad)
Step 2: d_ID = s * Q_ID computed
d_ID: (0bfec4e602cb1383d4fa34404dc33736ce4cf5634839585efff052157ebe2cd95245504d3a09556c5f525bd297c9637,0e9d965c497ed2396319a2672e1dab5a2dc8c65608a3ecc90ccd780be60caafabba68f7a6ac5bf6d2e126a5ce19d0313)
Extract complete - Private key generated for: bob@companyEU.com

=== Extract Algorithm ===
Generating private key for identity: alice@companyUS.com
Step 1: Q_ID = H1(ID) computed
Q_ID: (011c62e5b06ea5ee5268503323437440a94b54faaeaa3c01207c5c7f75adf71725ea0e3dec1ca13b7a82d0239079230f,0349b249fed7f4298ea7522c76705ac9e9873c9922658dbe9b6e2b276ca0bf441de2c76bd468f457c40382d95f695bfb)
Step 2: d_ID = s * Q_ID computed
d_ID: (0293ae1a38eb96a07ea3592055098b6c97b4fede6287043c732ca73bdec726a1b9ca84e8336f22c7ff3150bb4e7b7ff4,08813224797523aaca685141d633f502fec38aac2a3d30c08be39a873b88a3336d64c45b010b9eae555c7a6a0a7d99f)
Extract complete - Private key generated for: alice@companyUS.com

=====
--- Test Basic Encrypt/Decrypt ---
=====
Original message: "Hello Bob! This is a secret message."
Message length: 36 bytes
Message (hex): 48656c6cf20426f62212054686973206973206120736563726574206d6573736167652e

=== Encrypt Algorithm ===
Encrypting for: bob@companyEU.com
Encryption complete !

--- Ciphertext Components ---
U (G1 element): (0d769ff61e77db14288a7c80e00c1a10c809d64239a0b3c1ec1e5d6fa66aeba2e505656a46e72710894a512ed436b6c8,0fe00750634a7d56d316ea57dff39c18edb46d23f93b6810ea5e5022034335d8887db6a6fa67db4654d1e7d98443f9fb)
V (32 bytes): 339123a061ba7e522ca1cab07bd822e4eab7559a01b15d300312d23c55bafaf9b
W (36 bytes): e87be41d5cf9fe3f6b8849db58a2e0258b9de90b0f58f90f7f9f3c5fb78e41fb6167652e
Total ciphertext size: 173 bytes

=== Decrypt Algorithm ===
Decrypting for: bob@companyEU.com
Decryption successful !

--- Decryption Result ---
Decrypted message: "Hello Bob! This is a secret message."
Decrypted (hex): 48656c6cf20426f62212054686973206973206120736563726574206d6573736167652e
```

Figure 1: *BF-IBE scheme functionality test results*

References

- [1] D. F. Aranha et al. *RELIC is an Efficient Library for Cryptography*. <https://github.com/relic-toolkit/relic>.
- [2] Diego Aranha et al. “Implementing Pairings at the 192-Bit Security Level”. In: Jan. 2013, pp. 177–195. ISBN: 978-3-642-36333-7. DOI: [10.1007/978-3-642-36334-4_11](https://doi.org/10.1007/978-3-642-36334-4_11).
- [3] Diego F. Aranha. *Introduction to Pairings*. [Online; accessed 25. May. 2025]. 2017. URL: <https://ecc2017.cs.ru.nl/slides/ecc2017school-aranha.pdf>.
- [4] Diego F. Aranha et al. “Faster Explicit Formulas for Computing Pairings over Ordinary Curves”. In: *Advances in Cryptology – EUROCRYPT 2011*. Vol. 6632. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 2011, pp. 48–68. DOI: [10.1007/978-3-642-20465-4_4](https://doi.org/10.1007/978-3-642-20465-4_4). URL: https://doi.org/10.1007/978-3-642-20465-4_4.
- [5] M. Bellare and P. Rogaway. *Random Oracles Are Practical: a Paradigm for Designing Efficient Protocols*. ACM Press, New York, 1993, pp. 62–73.
- [6] Dan Boneh. *Identity Based Encryption*. [Online; accessed 25. May. 2025]. 2023. URL: <https://csrc.nist.gov/csrc/media/Presentations/2023/stppa5-ibe/images-media/20230209-stppa5-Dan-Boneh--IBE.pdf>.
- [7] Dan Boneh and Xavier Boyen. “Secure Identity Based Encryption Without Random Oracles”. In: *Advances in Cryptology – EUROCRYPT 2004*. Vol. 3027. Lecture Notes in Computer Science. Springer, 2004, pp. 223–238. DOI: [10.1007/978-3-540-24676-3_14](https://doi.org/10.1007/978-3-540-24676-3_14). URL: https://doi.org/10.1007/978-3-540-24676-3_14.
- [8] Dan Boneh, Xavier Boyen, and Eu-Jin Goh. “Hierarchical Identity Based Encryption with Constant Size Ciphertext”. In: *Advances in Cryptology – EUROCRYPT 2005*. Vol. 3494. Lecture Notes in Computer Science. Springer, 2005, pp. 440–456.
- [9] Dan Boneh and Matthew Franklin. “Identity-Based Encryption from the Weil Pairing”. In: *SIAM Journal on Computing* 32.3 (2003), pp. 586–615. DOI: [10.1137/S0097539701398521](https://doi.org/10.1137/S0097539701398521). eprint: <https://doi.org/10.1137/S0097539701398521>. URL: <https://doi.org/10.1137/S0097539701398521>.
- [10] Dan Boneh, Craig Gentry, and Michael Hamburg. “Space-Efficient Identity Based Encryption Without Pairings”. In: *48th Annual IEEE Symposium on Foundations of Computer Science (FOCS 2007)*. Providence, RI, USA: IEEE Computer Society, 2007, pp. 647–657. DOI: [10.1109/FOCS.2007.37](https://doi.org/10.1109/FOCS.2007.37). URL: <https://doi.org/10.1109/FOCS.2007.37>.
- [11] Dan Boneh and Jonathan Katz. “Improved Efficiency for CCA-Secure Cryptosystems built using Identity-Based Encryption”. In: *RSA Conference on Topics in Cryptology – CT-RSA 2005 (preliminary version 2004)*. preliminary version 2004. Springer, 2005, pp. 87–103.
- [12] Dan Boneh et al. “Public-Key Encryption with Keyword Search”. In: *Advances in Cryptology – EUROCRYPT 2004*. Vol. 3027. Lecture Notes in Computer Science. Springer, 2004, pp. 506–522. URL: <https://crypto.stanford.edu/~dabo/pubs/papers/encsearch.pdf>.
- [13] Xavier Boyen, Qixiang Mei, and Brent Waters. “Direct Chosen-Ciphertext Security from Identity-Based Techniques”. In: *ACM Conference on Computer and Communications Security (CCS) 2005*. ACM, 2005, pp. 320–329. URL: <https://www.cs.cmu.edu/~goyal/ibe.pdf>.
- [14] Ran Canetti, Shai Halevi, and Jonathan Katz. “A Forward-Secure Public-Key Encryption Scheme”. In: *Advances in Cryptology – EUROCRYPT 2003*. Vol. 2656. Lecture Notes in Computer Science. Springer, 2003, pp. 255–271. URL: <http://eprint.iacr.org/2003/083>.
- [15] Ran Canetti, Shai Halevi, and Jonathan Katz. “Chosen-Ciphertext Security from Identity-Based Encryption”. In: *Advances in Cryptology – EUROCRYPT 2004*. Vol. 3027. Lecture Notes in Computer Science. Springer, 2004, pp. 207–222. DOI: [10.1007/978-3-540-24676-3_14](https://doi.org/10.1007/978-3-540-24676-3_14). URL: <https://www.cs.umd.edu/~jkatz/papers/id-cca.pdf>.
- [16] David Cash et al. “Bonsai Trees, or How to Delegate a Lattice Basis”. In: *Advances in Cryptology – EUROCRYPT 2010*. Vol. 6110. Lecture Notes in Computer Science. Describes the Bonsai Tree approach to hierarchical identity-based encryption (HIBE) from LWE. Berlin, Heidelberg: Springer, 2010, pp. 523–552. DOI: [10.1007/978-3-642-13190-5_27](https://doi.org/10.1007/978-3-642-13190-5_27). URL: https://doi.org/10.1007/978-3-642-13190-5_27.
- [17] Ivan Damgård. *An Introduction to Cryptography*. 2024.
- [18] T. Elgamal. “A public key cryptosystem and a signature scheme based on discrete logarithms”. In: *IEEE Transactions on Information Theory* 31.4 (1985), pp. 469–472. DOI: [10.1109/TIT.1985.1057074](https://doi.org/10.1109/TIT.1985.1057074).
- [19] Armando Faz-Hernandez et al. *Hashing to Elliptic Curves*. RFC 9380. Aug. 2023. DOI: [10.17487/RFC9380](https://doi.org/10.17487/RFC9380). URL: <https://www.rfc-editor.org/info/rfc9380>.

- [20] E. Fujisaki and T. Okamoto. “Secure Integration of Asymmetric and Symmetric Encryption Schemes”. In: *Advances in Cryptology – CRYPTO ’99*. Vol. 1666. Lecture Notes in Computer Science. Springer, 1999, pp. 537–554.
- [21] Rosario Gennaro et al. “Secure Distributed Key Generation for Discrete-Log Based Cryptosystems”. In: *Advances in Cryptology – EUROCRYPT ’99*. Ed. by Jacques Stern. Berlin, Heidelberg: Springer Berlin Heidelberg, 1999, pp. 295–310. ISBN: 978-3-540-48910-8.
- [22] Craig Gentry. “Practical Identity-Based Encryption Without Random Oracles”. In: *Advances in Cryptology – EUROCRYPT 2006*. Vol. 4004. Lecture Notes in Computer Science. Springer, 2006, pp. 445–464. DOI: [10.1007/11761679_27](https://doi.org/10.1007/11761679_27). URL: https://doi.org/10.1007/11761679_27.
- [23] Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan. “Trapdoors for Hard Lattices and New Cryptographic Constructions”. In: *Proceedings of the 40th Annual ACM Symposium on Theory of Computing (STOC 2008)*. New York, NY, USA: ACM, 2008, pp. 197–206. DOI: [10.1145/1374376.1374407](https://doi.org/10.1145/1374376.1374407). URL: <https://doi.org/10.1145/1374376.1374407>.
- [24] Craig Gentry and Alice Silverberg. “Hierarchical ID-Based Cryptography”. In: *Advances in Cryptology – ASIACRYPT 2002*. Vol. 2501. Lecture Notes in Computer Science. London, UK: Springer-Verlag, 2002, pp. 548–566. DOI: [10.1007/3-540-36178-2_34](https://doi.org/10.1007/3-540-36178-2_34). URL: https://doi.org/10.1007/3-540-36178-2_34.
- [25] Matthew Green and Susan Hohenberger. *Universally Composable Adaptive Oblivious Transfer*. Tech. rep. 2008/163. Cryptology ePrint Archive, 2008. URL: <https://eprint.iacr.org/2008/163>.
- [26] Antoine Joux and Kim Nguyen. “Separating Decision Diffie–Hellman from Computational Diffie–Hellman in Cryptographic Groups”. In: *Journal of Cryptology* 16.4 (2003), pp. 239–247. ISSN: 1432-1378. DOI: [10.1007/s00145-003-0052-4](https://doi.org/10.1007/s00145-003-0052-4). URL: <https://doi.org/10.1007/s00145-003-0052-4>.
- [27] Ben Lynn. “On the Implementation of Pairing-based Cryptosystems”. PhD Dissertation. <https://crypto.stanford.edu/pbc/thesis.pdf>. PhD thesis. Stanford University, June 2007.
- [28] Yumi Sakemi, Satoru Kanno, and Riad S. Wahby. *Pairing-Friendly Curves*. Internet-Draft draft-irtf-cfrg-pairing-friendly-curves-12. Work in Progress. Internet Engineering Task Force, Nov. 2025. 54 pp. URL: <https://datatracker.ietf.org/doc/draft-irtf-cfrg-pairing-friendly-curves/12/>.
- [29] Adi Shamir. “Identity-Based Cryptosystems and Signature Schemes”. In: *Advances in Cryptology*. Ed. by George Robert Blakley and David Chaum. Berlin, Heidelberg: Springer Berlin Heidelberg, 1985, pp. 47–53. ISBN: 978-3-540-39568-3.
- [30] *The MIRACL Core Cryptographic Library*. [Online; accessed 13. November. 2025]. URL: <https://github.com/miracl/core/tree/master/java>.
- [31] Riad S. Wahby and Dan Boneh. *Fast and simple constant-time hashing to the BLS12-381 elliptic curve*. Cryptology ePrint Archive, Paper 2019/403. 2019. DOI: [10.13154/tches.v2019.i4.154-179](https://doi.org/10.13154/tches.v2019.i4.154-179). URL: <https://eprint.iacr.org/2019/403>.
- [32] Brent Waters. “Efficient Identity Based Encryption Without Random Oracles”. In: *Advances in Cryptology – EUROCRYPT 2005*. Vol. 3494. Lecture Notes in Computer Science. Springer, 2005, pp. 114–127. DOI: [10.1007/11426639_7](https://doi.org/10.1007/11426639_7). URL: https://doi.org/10.1007/11426639_7.

A Test results for Boneh-Franklin based IBE scheme

```
=====
TESTING COMPLETE ENCRYPT/DECRYPT CYCLE (MIRACL-CORE)
=====

=== Extract Algorithm ===
Generating private key for identity: bob@companyEU.com
Step 1: Q_ID = H1(ID) computed
Q_ID: (009e9db06e3dcc7934d088bf37b8d78c05cce8ca1f24b91d4b38145f02b81e968fd14df98e76d152d4a88429a3852a42
Step 2: d_ID = s * Q_ID computed
d_ID: (0bfec4e602cb1303d4fa34404dc33736ce4cf5634839585efff052157ebe2dcd95245504d3a09556c5f525bd297c9637
Extract complete - Private key generated for: bob@companyEU.com

=== Extract Algorithm ===
Generating private key for identity: alice@companyUS.com
Step 1: Q_ID = H1(ID) computed
Q_ID: (011c62e5b06ea5ee5268503323437440a94b54faaeaa3c01207c5c7f75adf71725ea0e3dec1ca13b7a82d0239079230f
Step 2: d_ID = s * Q_ID computed
d_ID: (0293ae1a38eb96a07ea3592055098b6c97b4fedee6287043c732ca73bdec726a1b9ca84e8336f22c7ff3150bb4e7b7ff4
Extract complete - Private key generated for: alice@companyUS.com

=====
--- Test 1: Basic Encrypt/Decrypt ---
=====
Original message: "Hello Bob! This is a secret message."
Message length: 36 bytes
Message (hex): 48656c6c6f20426f62212054686973206973206120736563726574206d6573736167652e

=== Encrypt Algorithm ===
Encrypting for: bob@companyEU.com
Encryption complete

--- Ciphertext Components ---
U (G1 element): (0d769ff61e77db14280a7c80e80c1a10c809d64239a0b3c1ec1e5d6fa66aeba2e505656a46e72710894a51
V (32 bytes): 339123a061ba7e522ca1cab07bd822e4eab7559a01b15d300312d23c55bafa9b
W (36 bytes): e87be41d5cf9fe3f6b8849db58a2e0258b9de90b0f58f90f7f9f3c5fb78e41fb6167652e
Total ciphertext size: 173 bytes

=== Decrypt Algorithm ===
Decrypting for: bob@companyEU.com
Decryption successful

--- Decryption Result ---
Decrypted message: "Hello Bob! This is a secret message."
Decrypted (hex): 48656c6c6f20426f62212054686973206973206120736563726574206d6573736167652e

Messages match: true
Test 1: PASS

=====
--- Test 2: Wrong Key (should fail) ---
=====

=== Decrypt Algorithm ===
Decrypting for: alice@companyUS.com
Error: Verification failed | ciphertext tampered
Alice decrypts Bob's message: rejected
Test 2: PASS
```

```

=====
--- Test 3: Cross-Identity ---
=====

=== Encrypt Algorithm ===
Encrypting for: alice@companyUS.com
Encryption complete

Attempt 1: Alice decrypts (should work)

=== Decrypt Algorithm ===
Decrypting for: alice@companyUS.com
Decryption successful
Alice's result: Secret message tailored for Alice

Attempt 2: Bob decrypts (should fail)

=== Decrypt Algorithm ===
Decrypting for: bob@companyEU.com
Error: Verification failed | ciphertext tampered
Bob's result: rejected

Alice can decrypt: true
Bob cannot decrypt: true
Test 3: PASS

=====
Test 4: Multiple Encryptions Same Message
=====

=== Encrypt Algorithm ===
Encrypting for: bob@companyEU.com
Encryption complete

=== Decrypt Algorithm ===
Decrypting for: bob@companyEU.com
Decryption successful
Encryption 1: OK

=== Encrypt Algorithm ===
Encrypting for: bob@companyEU.com
Encryption complete

=== Decrypt Algorithm ===
Decrypting for: bob@companyEU.com
Decryption successful
Encryption 2: OK

=== Encrypt Algorithm ===
Encrypting for: bob@companyEU.com
Encryption complete

=== Decrypt Algorithm ===
Decrypting for: bob@companyEU.com
Decryption successful
Encryption 3: OK

=== Encrypt Algorithm ===
Encrypting for: bob@companyEU.com
Encryption complete

```

=== Decrypt Algorithm ===
Decrypting for: bob@companyEU.com
Decryption successful
Encryption 4: OK

=== Encrypt Algorithm ===
Encrypting for: bob@companyEU.com
Encryption complete

=== Decrypt Algorithm ===
Decrypting for: bob@companyEU.com
Decryption successful
Encryption 5: OK
All ciphertexts unique: true
All decrypt correctly: true
Test 4: PASS

=====

TEST SUMMARY

=====

Test 1 - Basic Encrypt/Decrypt:	PASS
Test 2 - Wrong Key Rejection:	PASS
Test 3 - Cross-Identity:	PASS
Test 4 - Multiple Enc Same Message:	PASS

=====

ALL TESTS PASSED

=====

B Equivalent transformations for pairings

- $e(P, S + T) = e(P, S)e(P, T)$, where $P \in \mathbb{G}_1, S, T \in \mathbb{G}_2$
- $e(aP, bQ) = e(bP, aQ)$, where $a, b \in \mathbb{Z}_n$ and $P \in \mathbb{G}_1, Q \in \mathbb{G}_2$
- $e(aP, bQ) = e(abP, Q)$, where $a, b \in \mathbb{Z}_n$ and $P \in \mathbb{G}_1, Q \in \mathbb{G}_2$
- $e(aP, bQ) = e(P, Q)^{ab}$, where $a, b \in \mathbb{Z}_n$ and $P \in \mathbb{G}_1, Q \in \mathbb{G}_2$
- $e(-P, Q) = e(P, Q)^{-1} = e(P, -Q)$, where $P \in \mathbb{G}_1, Q \in \mathbb{G}_2$
- $e(P, 0) = e(0, Q) = 1$, where $P \in \mathbb{G}_1, Q \in \mathbb{G}_2$

C Hash-to-curve parameters for curve BLS12-381

In Table 2 encoding type denotes either random oracle (`hash_to_curve`) or nonuniform (`encode_to_curve`). E denotes the target elliptic curve over a field $\mathbb{F}$, p the characteristic of the field $\mathbb{F}$, m the extension degree of the field $\mathbb{F}$.

k , the target security level of the suite in bits. L the length parameter for `hash_to_field`, `expand_message` denotes one of the variants specified in Section 5.3 of [19] plus any parameters required for the specified variant (for example, H , the underlying hash function). f , a mapping function. h_{eff} is the scalar parameter for `clear_cofactor`. In addition the mapping f require additional parameters Z , M , `rational_map`, E' , and `iso_map`. [19]

BLS12-381 Curve parameter specification according to RFC9380		
	BLS12381G1_XMD:SHA-256_SSWU_RO_	BLS12381G2_XMD:SHA-256_SSWU_RO_
encoding type	<code>hash_to_curve</code>	<code>hash_to_curve</code>
E , elliptic curve	$E : y^2 = x^3 + 4$	$E : y^2 = x^3 + 4 \cdot (1 + \mathcal{I})$
E' , twisted curve equation	$y'^2 = x'^3 + A' \cdot x' + B'$	$y'^2 = x'^3 + A' \cdot x' + B'$
A' for E'	0x144698a3b8e9433d693a02c9 6d4982b0ea985383ee66a8d8e8 981aefd881ac98936f8da0e0f9 7f5cf428082d584c1d	$240 \cdot \mathcal{I}$
B' for E'	0x12e2908d11688030018b12e8 753eee3b2016c1f0f24f4070a0 b9c14fcef35ef55a23215a316c eaa5d1cc48e98e172be0	$1012 \cdot (1 + \mathcal{I})$
Base field $\mathbb{F}$ is $GF(p^m)$, where:		
p , prime	0x1a0111ea397fe69a4b1ba7b6434bacd7 64774b84f38512bf6730d2a0f6b0f624 1eabfffebf153ffffb9fefeffffffaaab	0x1a0111ea397fe69a4b1ba7b6434bacd7 64774b84f38512bf6730d2a0f6b0f624 1eabfffebf153ffffb9fefeffffffaaab
m	1	2
$(1, \mathcal{I})$ is the basis for $\mathbb{F}$, where $\mathbb{I}^2 + 1 = 0$ in $\mathbb{F}$		
k security parameter	128	128
expand message	<code>expand_message_xmd</code>	<code>expand_message_xmd</code>
H , hash function	$SHA - 256$	$SHA - 256$
L	64	64
f	Simplified SWU for $AB = 0$	Simplified SWU for $AB = 0$
Z	11	$-(2 + \mathcal{I})$
iso map, 11-isogeny map from E' to E	Appendix E.2 of [19]	Appendix E.3 of [19]
h_{eff} , compatible with the fast cofactor clearing method described by Scott [31]	0xd201000000010001	0xbc69f08f2ee75b3584c6a0ea91b35288 8e2a8e9145ad7689986ff031508ffe13 29c2f178731db956d82bf015d1212b02 ec0ec69d7477c1ae954cbc06689f6a35 9894c0adebbf6b4e8020005aaa95551
Simplified SWU mapping to curve E' isogenous to BLS12-318	Appendix F.2 of [19]	Appendix F.2 of [19]

Table 2: Overview of Identity-Based cryptographic construction families [6]